

# Spring Boot — Day 3 Complete Interview Notes

**Date: 8 September 2026 | 7:00–9:00 AM**

**Project: Staff REST API**

Today you extended your Day 2 Staff CRUD API by making it more **production-like** with:

- Input validation
- @Valid
- @NotBlank
- Exception handling
- orElseThrow()
- @ControllerAdvice
- @ExceptionHandler
- ResponseEntity
- HTTP status codes
- Proper status codes for POST, PUT, DELETE
- Handling missing Staff during GET/PUT

Your Day 2 foundation was the Controller → Service → Repository → Spring Data JPA → Hibernate → MySQL architecture.

Day 2 Spring Data JPA

---

## 1. What did we learn today?

The main goal was:

**Take a basic CRUD REST API and make its responses and error handling more professional.**

Before Day 3:

```
Client
↓
Controller
↓
Service
↓
Repository
↓
MySQL
```

After Day 3:

```
Client / Postman
↓
Controller
↓
@RequestBody
↓
@Valid
↓
Validation
↓
Service
↓
Repository
↓
Spring Data JPA
↓
Hibernate
↓
MySQL
```

For errors:

```
Service
↓
orElseThrow()
↓
Exception
↓
@ControllerAdvice
↓
```

@ExceptionHandler

↓

HTTP Error Response

---

## 2. Input Validation

### What is validation?

Validation means checking whether the incoming data satisfies the rules defined by the application.

For example, suppose Staff requires:

name → cannot be blank  
department → cannot be blank  
role → cannot be blank

We don't want this:

JSON

```
{  
  "name": "",  
  "department": "Computer",  
  "role": "Developer"  
}
```

to reach the Service layer.

---

## 3. @NotBlank

You used:

Java

```
@NotBlankprivateStringname;
```

### What does @NotBlank do?

It validates a String and rejects:

```
null
""
" "
```

But:

```
"Aniket" ✓
```

is valid.

You applied it to your Staff entity:

Java

```
@NotBlankprivateStringname;
```

```
@NotBlankprivateStringdepartment;
```

```
@NotBlankprivateStringrole;
```

## Interview answer

**"@NotBlank is a validation constraint used for String fields. It ensures that the value is not null, empty, or whitespace-only."**

---

## 4. @Valid

You added:

Java

```
@PostMapping("/api/staff")
publicResponseEntity<Staff> addStaff(
    @Valid @RequestBodyStaffstaff) {

    StaffsavedStaff=staffService.addStaff(staff);

    returnResponseEntity.status(HttpStatus.CREATED)
        .body(savedStaff);
}
```

## What does @Valid do?

@Valid **triggers validation** of the Java object according to the validation constraints defined on its fields.

For example:

Java

```
@NotBlankprivateStringname;
```

and:

Java

```
@Valid @RequestBodyStaffstaff
```

means Spring will check the name constraint when processing the request.

## Interview answer

**"@Valid triggers validation of the object received in the request. It checks the validation constraints defined on its fields, such as @NotBlank, before the request proceeds to the Service layer."**

---

## 5. @NotBlank VS @Valid

This is an important interview question.

@NotBlank

Defines **the validation rule**.

Java

```
@NotBlankprivateStringname;
```

@Valid

Triggers **the validation process**.

Java

```
@Valid @RequestBodyStaffstaff
```

## Easy memory

@NotBlank

↓

WHAT rule?

@Valid

↓

TRIGGER the rules

---

## 6. Validation Request Flow

Suppose Postman sends:

JSON

```
{  
  "name": "",  
  "department": "Computer",  
  "role": "Developer"  
}
```

Flow:

JSON

↓

@RequestBody

↓

Staff Java Object

↓

@Valid

↓

@NotBlank

↓

name = ""

↓

 Validation fails

↓

Request rejected

↓

400 Bad Request

The important point:

**If validation fails, the request doesn't proceed to the Service layer.**

---

## 7. Exception Handling

Suppose we request:

```
GET /api/staff/9999
```

but Staff 9999 doesn't exist.

We don't want:

```
null
```

We want a meaningful error:

```
404 Not Found  
Staff Not Found
```

---

## 8. Optional and `orElseThrow()`

Your Repository's:

Java

```
findById(id)
```

returns an Optional.

You changed your Service to:

Java

```
public Staff getStaffById(int id) {  
  
    return staffRepository.findById(id)  
        .orElseThrow() ->  
            new RuntimeException("Staff Not Found");  
}
```

## What does `orElseThrow()` do?

If the Staff exists:

Optional contains Staff



return Staff

If it doesn't exist:

Optional.empty()



orElseThrow()



Exception

## Interview answer

**"orElseThrow() returns the value when it is present, but throws a specified exception when the Optional is empty. We use it to handle cases where the requested Staff does not exist."**

---

## 9. Very Important Distinction

Remember:

`orElseThrow()`



THROWS the exception

while:

`@ExceptionHandler`



HANDLES the exception

and:

`@ControllerAdvice`



CENTRALIZES exception handling

This distinction is highly useful in interviews.

---

## 10. @ExceptionHandler

You created a global handler similar to:

Java

```
@ExceptionHandler(RuntimeException.class)
public ResponseEntity<String> handleException(RuntimeException ex) {

    return ResponseEntity.status(404)
        .body(ex.getMessage());
}
```

What does @ExceptionHandler do?

It specifies how a particular exception should be handled.

Interview answer

**"@ExceptionHandler is used to handle a particular exception and return an appropriate response, such as an HTTP status code and response body."**

---

## 11. @ControllerAdvice

Your global exception class uses:

Java

```
@ControllerAdvice
public class GlobalExceptionHandler {

    ...
}
```

What does it do?

@ControllerAdvice allows exception handling to be **centralized across controllers**.

Instead of putting exception-handling code inside every controller, we can put it in one global handler.

## Interview answer

**"@ControllerAdvice is a class-level annotation used to centralize exception handling across multiple controllers. It is commonly used with @ExceptionHandler to provide global exception handling."**

---

## 12. @ControllerAdvice VS @ExceptionHandler

Very important interview question.

@ExceptionHandler

WHAT exception should I handle?

@ControllerAdvice

WHERE should the handling be centralized?

Easy memory:

@ControllerAdvice

↓

Global handling

@ExceptionHandler

↓

Specific exception handling

---

## 13. Complete Exception Flow

For:

GET /api/staff/9999

when the Staff doesn't exist:

Postman

↓

GET request

↓

StaffController

↓

```
StaffService
↓
Repository.findById(9999)
↓
Optional.empty()
↓
orElseThrow()
↓
RuntimeException
↓
@ControllerAdvice
↓
@ExceptionHandler
↓
404 Not Found
↓
"Staff Not Found"
```

You actually tested this successfully with Postman. 

---

## 14. ResponseEntity

Today you learned how to control the HTTP response.

Instead of simply:

Java

```
public Staff addStaff(...)
```

you changed it to:

Java

```
public ResponseEntity<Staff> addStaff(...)
```

### What does ResponseEntity provide?

It allows you to control:

```
HTTP status
Response body
Headers
```

## Interview answer

**"ResponseEntity is used to control the HTTP response, including the status code and response body. In our Staff API, we used it to return appropriate HTTP status codes for different CRUD operations."**

---

## 15. POST → 201 Created

Your POST now uses:

Java

```
@PostMapping("/api/staff")
public ResponseEntity<Staff> addStaff(
    @Valid @RequestBody Staff staff) {

    Staff savedStaff = staffService.addStaff(staff);

    return ResponseEntity.status(HttpStatus.CREATED)
        .body(savedStaff);
}
```

Successful response:

201 Created

with:

JSON

```
{
  "id": 7,
  "name": "Mahesh",
  "department": "Computer",
  "role": "Developer"
}
```

## Why 201?

Because a new resource was created.

## Interview answer

**"201 Created indicates that a new resource was successfully created."**

---

## 16. PUT → 200 OK

Your PUT is:

Java

```
@PutMapping("/api/staff/{id}")
public ResponseEntity<Staff> updateStaff(
    @PathVariable int id,
    @RequestBody Staff staff) {

    return ResponseEntity.ok(
        staffService.updateStaff(id, staff)
    );
}
```

Successful response:

200 OK

with the updated Staff object.

`ResponseEntity.ok()`

Java

`ResponseEntity.ok(data)`

means:

200 OK

+

data

---

## 17. DELETE → 204 No Content

Your DELETE is:

Java

```
@DeleteMapping("/api/staff/{id}")
public ResponseEntity<Void> deleteStaff(
    @PathVariable int id) {

    staffService.deleteStaff(id);

    return ResponseEntity.noContent().build();
}
```

Successful response:

204 No Content

There is no response body.

### Interview answer

**"204 No Content indicates that the request was successfully processed but there is no response body to return."**

---

## 18. HTTP Status Codes

Memorize these:

| Code                             | Meaning                 | Staff API       |
|----------------------------------|-------------------------|-----------------|
| <b>200 OK</b>                    | Successful request      | GET / PUT       |
| <b>201 Created</b>               | Resource created        | POST            |
| <b>204 No Content</b>            | Success, no body        | DELETE          |
| <b>400 Bad Request</b>           | Invalid request/input   | Validation      |
| <b>404 Not Found</b>             | Resource doesn't exist  | Staff not found |
| <b>500 Internal Server Error</b> | Unexpected server error | Server problem  |

### Memory trick

200 → OK

201 → Created

204 → Nothing

400 → Bad request  
404 → Not found  
500 → Server problem

---

## 19. Your CRUD Status Code Design

Your Staff API now follows:

CREATE  
POST  
↓  
201 Created

READ  
GET  
↓  
200 OK

UPDATE  
PUT  
↓  
200 OK

DELETE  
DELETE  
↓  
204 No Content

NOT FOUND  
↓  
404 Not Found

INVALID INPUT  
↓  
400 Bad Request

---

## 20. Update Exception Handling

Initially your update method was:

Java

```
StaffexistingStaff=staffRepository.findById(id).orElse(null);

if (existingStaff!=null) {
    ...
}

returnnull;
```

We improved it to:

Java

```
publicStaffupdateStaff(intid, Staffstaff) {

    StaffexistingStaff=staffRepository.findById(id)
        .orElseThrow() ->
        newRuntimeException("Staff Not Found");

    existingStaff.setName(staff.getName());
    existingStaff.setDepartment(staff.getDepartment());
    existingStaff.setRole(staff.getRole());

    returnstaffRepository.save(existingStaff);
}
```

**Why remove if (existingStaff != null)?**

Because:

Java

```
orElseThrow()
```

guarantees:

Staff exists → object returned  
Staff doesn't exist → exception thrown

Therefore:

Java

```
if (existingStaff!=null)
```

is unnecessary.

## Interview answer

**"orElseThrow() guarantees that the method either returns a Staff object or throws an exception, so an additional null check is unnecessary."**

---

## 21. Complete POST Flow

When you send:

POST /api/staff

with:

JSON

```
{  
  "name": "Aniket",  
  "department": "Computer",  
  "role": "Developer"  
}
```

the flow is:

```
Postman  
↓  
HTTP POST  
↓  
Controller  
↓  
@RequestBody  
↓  
Staff Java Object  
↓  
@Valid  
↓  
@NotBlank validation  
↓  
Service  
↓  
Repository.save()
```

↓  
Spring Data JPA  
↓  
Hibernate  
↓  
MySQL  
↓  
Record stored  
↓  
201 Created  
↓  
Postman

The underlying CRUD request architecture is consistent with your Day 2 implementation.

Day 2 Spring Data JPA

---

## 22. Complete GET Not Found Flow

GET /api/staff/9999  
↓  
Controller  
↓  
Service  
↓  
Repository.findById()  
↓  
Optional.empty()  
↓  
orElseThrow()  
↓  
RuntimeException  
↓  
@ControllerAdvice  
↓  
@ExceptionHandler  
↓  
404 Not Found

---

## 23. Complete PUT Flow

For:

PUT /api/staff/7

with:

JSON

```
{  
  "name": "Ram",  
  "department": "Computer",  
  "role": "Analyst"  
}
```

PUT request

↓

Controller

↓

@PathVariable → 7

↓

@RequestBody → Staff data

↓

Service

↓

findById(7)

↓

Staff found

↓

Update fields

↓

Repository.save()

↓

MySQL

↓

ResponseEntity.ok()

↓

200 OK

---

## 24. Complete DELETE Flow

For:

DELETE /api/staff/6

DELETE request



Controller



@PathVariable → 6



Service



Repository



MySQL



Record deleted



ResponseEntity.noContent()



204 No Content

---

## 25. Most Important Interview Questions

### Q1. What is @Valid?

@Valid triggers validation of the incoming Java object according to the validation constraints defined on its fields.

---

### Q2. What is @NotBlank?

@NotBlank validates a String and ensures it is not null, empty, or whitespace-only.

---

### Q3. Difference between @Valid and @NotBlank?

@NotBlank defines a validation constraint, while @Valid triggers validation of the object's constraints.

---

### Q4. What is orElseThrow()?

orElseThrow() returns the value if the Optional contains one; otherwise, it throws the specified exception.

---

### Q5. Does orElseThrow() handle the exception?

No. orElseThrow() throws the exception. The exception can then be handled by an exception handler.

---

### Q6. What is @ExceptionHandler?

It is used to handle a particular exception and return an appropriate HTTP response.

---

### Q7. What is @ControllerAdvice?

It is used to centralize exception handling across multiple controllers.

---

### Q8. Difference between @ControllerAdvice and @ExceptionHandler?

@ControllerAdvice provides centralized/global exception handling, while @ExceptionHandler defines how a particular exception is handled.

---

### Q9. What is ResponseEntity?

ResponseEntity allows us to control the HTTP response, including the status code, body, and headers.

---

## Q10. Why use 201 Created for POST?

Because POST successfully creates a new resource.

---

## Q11. Why use 200 OK for PUT?

Because the update operation completed successfully and we can return the updated resource.

---

## Q12. Why use 204 No Content for DELETE?

Because the deletion was successful and there is no response body to return.

---

## Q13. What happens when a Staff ID doesn't exist?

findById() returns an empty Optional, orElseThrow() throws an exception, and the global exception handler handles it and returns 404 Not Found.

---

## 26. ★ One-Minute Day 3 Revision

Before an interview, remember this:

@Valid

= Triggers validation

@NotBlank

= String cannot be null/empty/whitespace

orElseThrow()

= Throws when Optional is empty

@ExceptionHandler

= Handles exception

@ControllerAdvice

= Centralized/global exception handling

ResponseEntity  
= Controls HTTP response

201  
= Created

200  
= OK

204  
= No Content

400  
= Bad Request

404  
= Not Found

500  
= Server Error

---

## Day 3 Interview-Level Architecture Answer

If the interviewer asks:

**"Explain how you improved your Spring Boot Staff API."**

Say:

**"I started with a Controller-Service-Repository based Staff REST API. On Day 3, I added input validation using @Valid and field-level constraints such as @NotBlank. I also added exception handling using orElseThrow(), @ExceptionHandler, and @ControllerAdvice so that missing resources return an appropriate 404 response. Finally, I used ResponseEntity to control HTTP responses and return proper REST status codes, such as 201 Created for POST, 200 OK for PUT, and 204 No Content for DELETE."**

 **That is a strong fresher/junior-level Spring Boot interview answer.**

## ✓ Day 3 Status

### Spring Boot REST API

|   |                   |   |
|---|-------------------|---|
|   |                   |   |
| — | Validation        | ✓ |
| ✓ | Valid             | □ |
| ✓ | NotBlank          | □ |
| ✓ | ExceptionHandler  | □ |
| ✓ | ElseThrow()       | □ |
| ✓ | ExceptionHandler  | □ |
| ✓ | ControllerAdvice  | □ |
| ✓ | ResponseEntity    | □ |
| ✓ | HTTP Status Codes | □ |
| — | POST → 201        | ✓ |
| — | PUT → 200         | ✓ |
| — | DELETE → 204      | ✓ |
| — | 404 Not Found     | ✓ |

### Day 3 Spring Boot: COMPLETE 🎯

Tomorrow, we should build on this rather than restart basics.

Sources